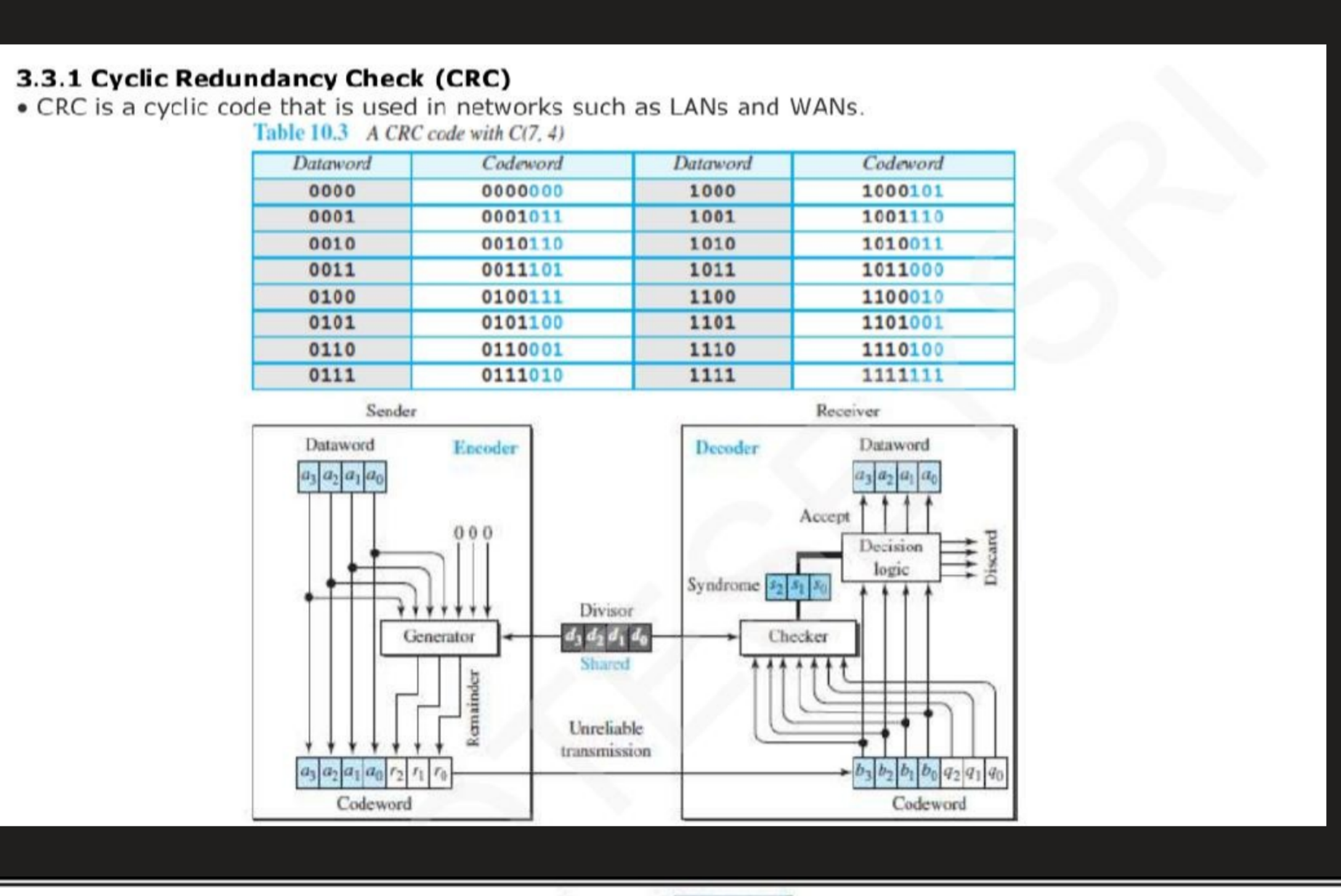
Thursday, January 2, 2025 10:19 PM



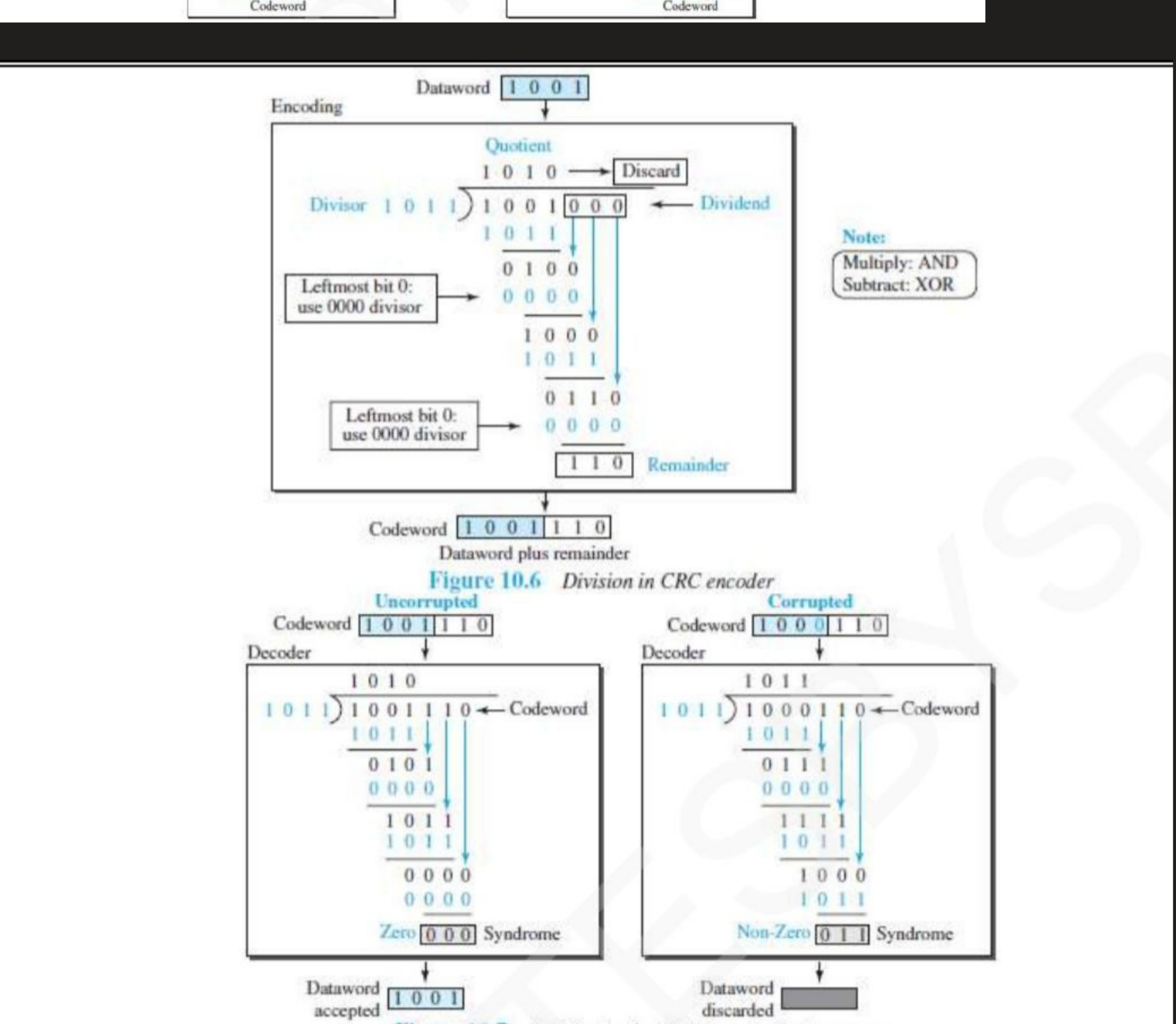
3.3.1 Cyclic Redundancy Check (CRC)

- CRC is a cyclic code that is used in networks such as LANs and WANs.

Table 10.3 *A CRC code with C(7, 4)*

Dataword	Codeword	Dataword	Codeword
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Sender
Receiver
Dataword
Encoder
a_3 a_2 a_1 a_0
000
Generator
Remainder
a_3 a_2 a_1 a_0 r_2 r_1 r_0
Codeword
Divisor
d_3 d_2 d_1 d_0
Shared
Unreliable transmission
Decoder
Dataword
a_3 a_2 a_1 a_0
Accept
Decision logic
Discard
Syndrome s_2 s_1 s_0
Checker
b_3 b_2 b_1 b_0 q_2 q_1 q_0
Codeword



Encoding
Dataword 1 0 0 1
Quotient
1 0 1 0 → Discard
Divisor 1 0 1 1
1 0 0 1 0 0 0 ← Dividend
1 0 1 1
0 1 0 0
Leftmost bit 0: use 0000 divisor
0 0 0 0
1 0 0 0
1 0 1 1
0 1 1 0
Leftmost bit 0: use 0000 divisor
0 0 0 0
1 1 0 Remainder
Codeword 1 0 0 1 1 1 0
Dataword plus remainder
Note: Multiply: AND Subtract: XOR

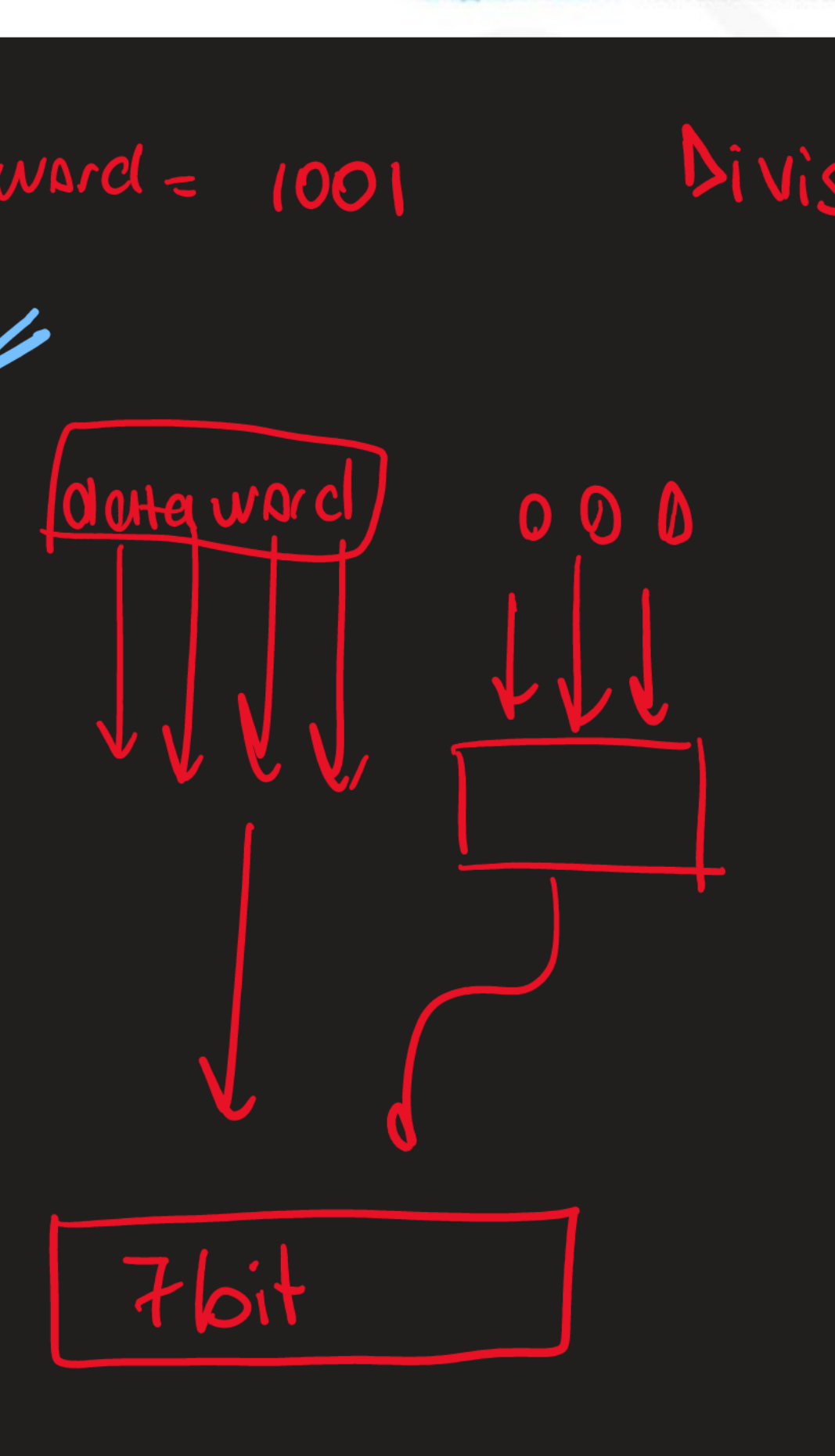
Figure 10.6 *Division in CRC encoder*

Uncorrupted
Codeword 1 0 0 1 1 1 0
Decoder
1 0 1 0
1 0 1 1) 1 0 0 1 1 1 0 ← Codeword
1 0 1 1
0 1 0 1
0 0 0 0
1 0 1 1
1 0 1 1
0 0 0 0
0 0 0 0
Zero 0 0 0 Syndrome
Dataword accepted 1 0 0 1
Corrupted
Codeword 1 0 0 0 1 1 0
Decoder
1 0 1 1
1 0 1 1) 1 0 0 0 1 1 0 ← Codeword
1 0 1 1
0 1 1 1
0 0 0 0
1 1 1 1
1 0 1 1
1 0 0 0
1 0 1 1
Non-Zero 0 1 1 Syndrome
Dataword discarded

Figure 10.7 *Division in the CRC decoder for two cases*

data word = 1001   Divisior = 1011

gönderici



data word
000
7bit

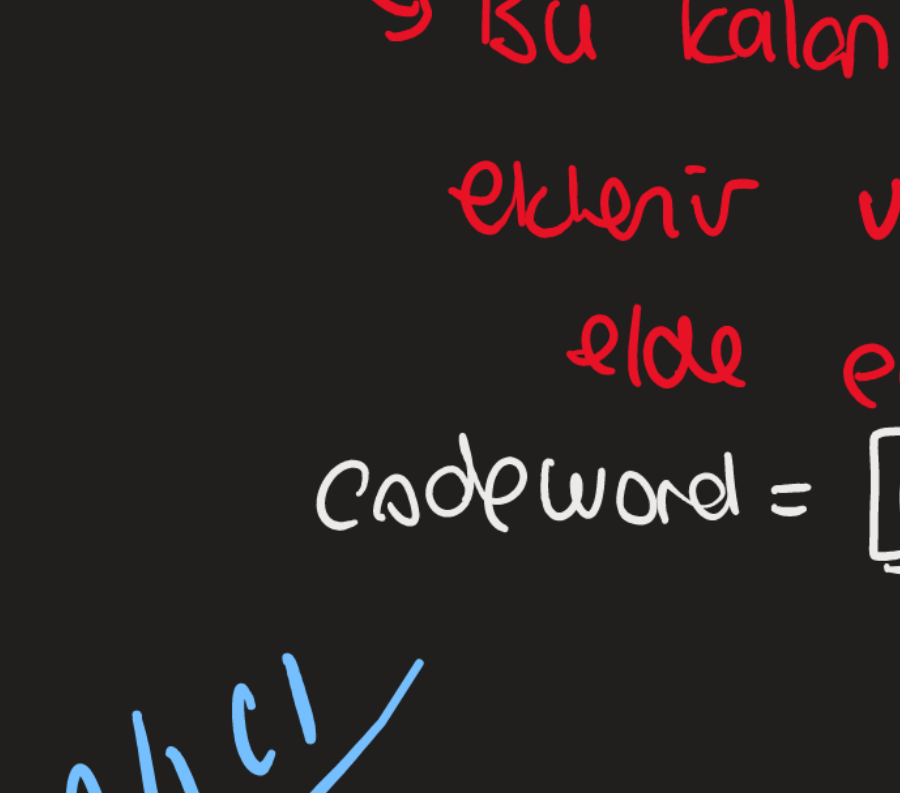
```
1001000 | 1011
1011    | 1010
 0100
 0000
  1000
  1011
  00110
   0000
   110 → kalan
```

Bu kalan dataword'e eklenir ve codeword elde edilir

data word + remain = codeword

codeword = 1001110

Alıcı



divisior
syndrome oluşturur
checker
7bit

bu syndrome 000 ise sorun yoktur.

```
1001110 | 1011
1011    | 1010
00101
 0000
  1011
  1011
  00000
   0000
   000 → Kalan 000
```

olduğundan alınan codeword'de sorun yoktur.

→ Şimdi kendimiz bir örnek oluşturalım.

Divisior = 1010 olsun

Dataword = 1001 olsun

gönderici için:

```
1001000 | 1010
1010    | 1011
00110
 0000
  1100
  1010
  01100
   1010
   0110 → bu dataword'e eklenir
```

codeword = 1001110 olur.

alıcı için:

```
1001110 | 1010
1010    | 1011
0011
 0000
  1111
  1010
   1010
   1010
    000
```

Kalan 0 olduğundan codeword'de sorun yok

değişti

codeword : 1001100 olursa?

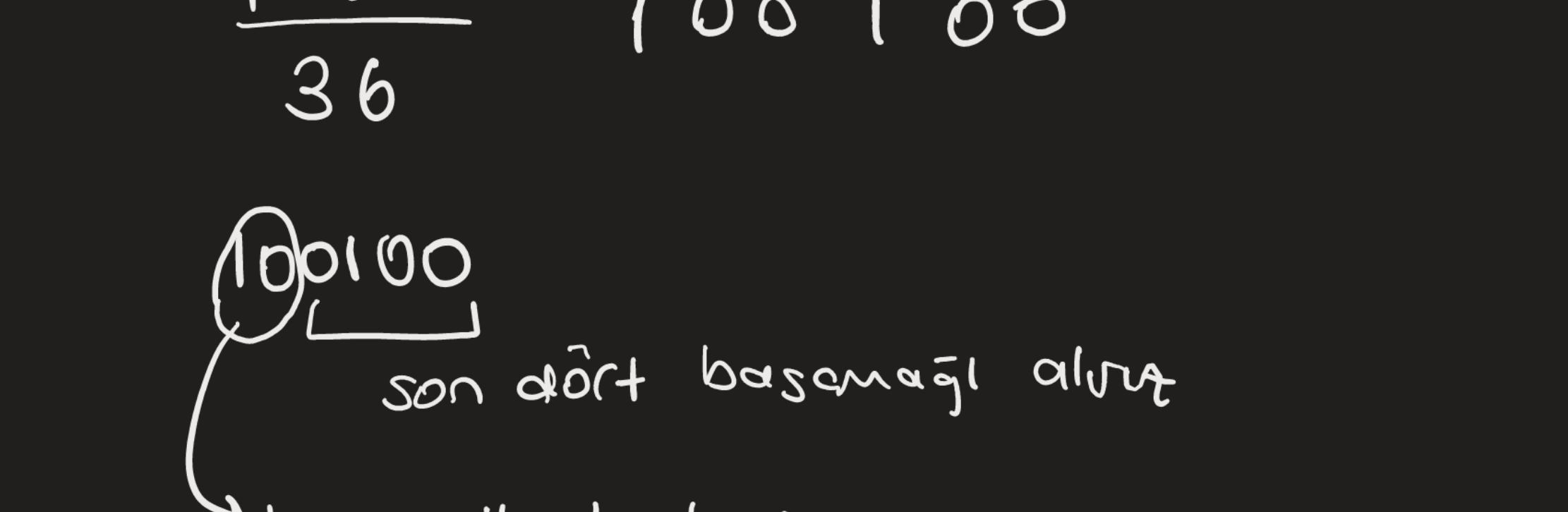
```
1001100 | 1010
1010    | 1011
00111
 0000
  1110
  1010
   1000
   1010
    010 → kala 000
```

değil

codeword hatalıdır. #

Gönderici (Sender) tarafında:
* Tüm veri parçacıklarının sayısal değerleri toplanır.
* Bu toplamın son 4 basamağı alınır (4 bitlik bir sayı elde edilir).
* Bu 4 bitlik sayının tamamlayıcı değeri bulunur (örneğin, 1011'in tamamlayıcısı 0100'dür).
* Bu tamamlayıcı değer, orijinal veri paketine eklenir ve gönderilir.

* Alıcı (Receiver) tarafında:
* Gelen veri paketindeki tüm sayısal değerler ve eklenen tamamlayıcı değer toplanır.
* Bu toplamın son 4 basamağı alınır.
* Eğer bu sonuç 0 ise, veri paketinde herhangi bir hata olmadığı anlamına gelir.
* Eğer sonuç 0 değilse, veri paketinde bir hata olduğu anlamına gelir ve paket atılır.



data is a list of five 4-bit numbers that we want to send to a destination.
ddition to sending these numbers, we send the sum of the numbers.
example:
Let set of numbers = (7, 11, 12, 0, 6).
We send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers.
receiver adds the five numbers and compares the result with the sum.
e result & the sum are the same,
The receiver assumes no error, accepts the five numbers, and discards the sum.
Otherwise, there is an error somewhere and the data are not accepted.

```
  7
 11
 12
  0
+ 6   1
-----
 36
```

32	16	8	4	2	1
2^5	2^4	2^3	2^2	2^1	2^0
1	0	0	1	0	0

100100 son dört basamağı alırız

bunun ile toplarız

```
  0100
+   10
  0110
```

bunun 1'e tümleyenini alırız

0110 → 1' → 1001 = ⑨

bu değeri veriler ile beraber göndeririz

gönderilen: 7, 11, 12 0, 6, 9

alıcı taraf =

```
  7
 11
 12
  0
  6
  9
+
 45
```

32	16	8	4	2	1
1	0	1	1	0	1

topla   4bit al

```
 1101
   10
 1111
```

1'e tümleyen = 0000

sonuç 0   o zaman hata yok demektir.